



DHIRUBAI AMBANI UNIVERSITY

A deep reinforcement learning based approach to automated Stock Trading using CNN and PPO

Student :

MEENAKSHI IYER (202418031),
RISHI BARAPATRE (202418008)

professor :

DR. ABHISHEK JINDAL



Contents

1	Introduction	2
2	Methodology	4
3	Data Acquisition	4
4	Feature Engineering	4
5	Preprocessing and Scaling	5
6	Reinforcement Learning Environment	7
7	Model Architecture	7
8	Training Procedure	9
8.1	Results and Observation	10
9	Future Work	12



Abstract

Algorithmic trading has evolved from simple rule-based systems to sophisticated models capable of learning directly from market dynamics. In this project, we explore a deep reinforcement learning framework for automated asset allocation using Proximal Policy Optimization (PPO) combined with a Convolutional Neural Network (CNN) feature extractor. Unlike traditional trading systems that rely on fixed indicators or static thresholds, our approach allows an agent to learn allocation decisions by interacting with a simulated market environment and maximizing cumulative portfolio returns under realistic constraints.

Using a dataset of 50 stocks, we engineered 250 technical features and constructed temporal windows that the CNN processes to extract meaningful patterns from historical price movements. The PPO agent, operating on these compressed representations, learns to output portfolio weights rather than discrete buy/sell actions making the system more aligned with practical multi-asset portfolio management. Our experiments demonstrate that this RL-based asset allocation approach can adapt to shifting market conditions and, in backtesting, achieves performance that is competitive with the NIFTY 50 benchmark. The results highlight the potential of combining CNN feature extraction with PPO to build scalable and data-driven allocation strategies.

1 Introduction

Financial markets today are increasingly shaped by automation, data-driven decision making, and rapid execution. Traditional rule-based trading strategies often built on indicators such as moving averages, RSI, or Bollinger Bands struggle to keep up with the complexity and speed of modern markets. As highlighted in the motivation behind this work, recent advances in deep learning and reinforcement learning have made it possible for trading systems to learn directly from market interactions rather than depending solely on handcrafted rules.

Reinforcement learning (RL), in particular, provides a natural framework for sequential decision-making problems such as trading and portfolio management. An RL agent observes the market state, selects an action, and receives a reward that reflects trading performance. Over time, it learns a policy that maximizes expected returns. Among various RL algorithms, Proximal Policy Optimization (PPO) has emerged as a stable and efficient choice for continuous-action financial environments because it prevents overly aggressive updates while still allowing flexible policy improvement.

Here the goal is not to issue buy/sell signals for a single stock but to optimally distribute capital across multiple assets. To represent the high dimensional feature space generated from 50 stocks and 250 technical indicators, we employ a Convolutional Neural Network (CNN) as the feature extractor. The CNN processes temporal windows of technical indicators, capturing local patterns and compressing them into a meaningful latent vector. This representation is then used by the PPO actor-critic architecture to output portfolio weights.

The workflow of our system—from data acquisition, feature engineering, and environment design to training and backtesting—follows the structure illustrated in the project documentation [?]. Although access to high-frequency data and multi-stock execution constraints posed challenges, our results show that a CNN-PPO combination can learn



adaptive allocation strategies that track and, in some periods, outperform a standard benchmark such as the NIFTY 50 index.

Overall, this project demonstrates how deep reinforcement learning can move beyond single-stock trading and toward more realistic, scalable asset allocation frameworks.

2 Methodology

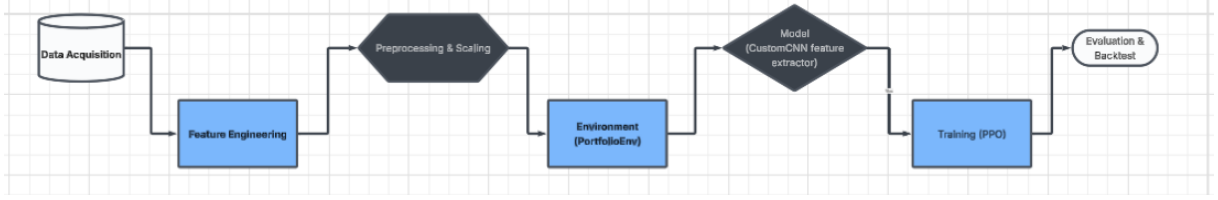


Figure 1: Workflow of the Deep Reinforcement Learning Asset Allocation System

3 Data Acquisition

A universe of 50 equities is selected and historical price data is downloaded using the utilities defined in `data_utils.py`. The configuration parameters, such as the date range, ticker list, window length, and benchmark index, are centralized in a file. To ensure reproducibility and efficiency, the data downloading procedure uses file-level caching: previously fetched CSV files are reused whenever available. Additionally, the NIFTY 50 index is downloaded as a benchmark time series.

4 Feature Engineering

For each stock, a set of technical indicators is generated using the functions provided in `technical_indicators.py`. These include the Relative Strength Index (RSI), Simple Moving Average (SMA), and Bollinger Band components (upper, middle, lower). With five indicators computed per stock, the full feature set consists of 250 engineered variables. These features are merged with the adjusted closing prices, forming a multi-asset technical dataset. Inspired by the workflow diagram in the project slides, temporal windows of length 30 are then created to capture short-term market dynamics. Each observation therefore becomes a 3D tensor shaped as

$$(\text{number of features}) \times (\text{window length}) \times 1,$$

suitable for convolutional processing.

Let $\{S_1, S_2, \dots, S_N\}$ denote the set of $N = 50$ assets. For each asset S_i , we consider its adjusted closing price time series

$$P_i(t), \quad t = 1, 2, \dots, T.$$

From this price series, we compute a set of $K = 5$ technical indicators defined in `technical_indicators.py`:

1. Relative Strength Index (RSI):

$$RSI_i(t) = 100 - \frac{100}{1 + RS_i(t)}, \quad RS_i(t) = \frac{\text{avg gain over window}}{\text{avg loss over window}}.$$

2. Simple Moving Average (SMA):

$$\text{SMA}_i(t) = \frac{1}{L} \sum_{\tau=t-L+1}^t P_i(\tau),$$

where L is the smoothing period.

3. Bollinger Bands:

$$\text{BB}_i^{\text{mid}}(t) = \text{SMA}_i(t), \quad \sigma_i(t) = \sqrt{\frac{1}{L} \sum_{\tau=t-L+1}^t (P_i(\tau) - \text{SMA}_i(t))^2},$$

$$\text{BB}_i^{\text{upper}}(t) = \text{SMA}_i(t) + k \sigma_i(t), \quad \text{BB}_i^{\text{lower}}(t) = \text{SMA}_i(t) - k \sigma_i(t),$$

where typically $k = 2$.

Thus, for each stock S_i , we construct a feature vector at time t :

$$\mathbf{x}_i(t) = [\text{RSI}_i(t), \text{SMA}_i(t), \text{BB}_i^{\text{upper}}(t), \text{BB}_i^{\text{mid}}(t), \text{BB}_i^{\text{lower}}(t)]^\top \in R^K.$$

Stacking these across the N assets yields a feature vector of dimension

$$\mathbf{x}(t) = [\mathbf{x}_1(t)^\top, \mathbf{x}_2(t)^\top, \dots, \mathbf{x}_N(t)^\top]^\top \in R^{NK} = R^{250}.$$

Temporal Window Construction

To capture short-term temporal dependencies, we construct rolling windows of length $W = 30$ time steps. For any current time t , the model observes the sequence:

$$\mathbf{X}(t) = [\mathbf{x}(t - W + 1), \mathbf{x}(t - W + 2), \dots, \mathbf{x}(t)].$$

Formally,

$$\mathbf{X}(t) \in R^{NK \times W}.$$

For compatibility with convolutional layers, this matrix is reshaped into a 3D tensor:

$$\mathcal{X}(t) \in R^{NK \times W \times 1}.$$

$$\mathcal{X}(t)[f, \tau, 1] = \text{value of feature } f \text{ at time } (t - W + \tau).$$

This tensor serves as the input to the CNN feature extractor. The convolutional kernels slide along the temporal dimension τ , enabling the extractor to detect local trends, volatility regimes, and short-term momentum patterns for each feature independently.

5 Preprocessing and Scaling

Once the raw prices and engineered technical indicators have been assembled into a unified multi-asset dataset, the next stage involves preprocessing to ensure consistency, numerical stability, and suitability for model training. This stage consists of data cleaning, temporal alignment, partitioning, and feature scaling.

Data Cleaning and Alignment

Because different assets may contain missing values due to holidays, suspension periods, or incomplete historical availability, the dataset is first aligned across all N assets by performing an inner join on timestamps. Formally, letting $\mathcal{T}_i$ denote the set of timestamps available for stock S_i , the valid trading dates are

$$\mathcal{T} = \bigcap_{i=1}^N \mathcal{T}_i.$$

All indicators and prices are restricted to the index set $\mathcal{T}$, ensuring that every observation contains a complete feature vector across all assets. Rows containing any residual missing values are dropped, yielding a clean matrix

$$\mathbf{X} \in R^{T' \times (NK+N)},$$

where T' is the number of aligned time steps, NK indicator features exist, and N price columns are retained for portfolio valuation.

Feature Scaling

The technical indicators (but not the price columns) are scaled to achieve numerical stability and accelerate gradient-based learning. Let $\mathbf{x}_{\text{sig}}(t) \in R^{NK}$ denote the vector of technical indicators at time t . The **StandardScaler** computes:

$$\mu = \frac{1}{T_{\text{train}}} \sum_{t=1}^{T_{\text{train}}} \mathbf{x}_{\text{sig}}(t), \quad \sigma = \sqrt{\frac{1}{T_{\text{train}}} \sum_{t=1}^{T_{\text{train}}} (\mathbf{x}_{\text{sig}}(t) - \mu)^2}.$$

Each signal feature is then standardized according to:

$$\tilde{\mathbf{x}}_{\text{sig}}(t) = \frac{\mathbf{x}_{\text{sig}}(t) - \mu}{\sigma}.$$

Importantly, the parameters (μ, σ) are *fitted only on the training set*, and the same values are applied to the test set:

$$\tilde{\mathbf{x}}_{\text{sig}}^{\text{test}}(t) = \frac{\mathbf{x}_{\text{sig}}^{\text{test}}(t) - \mu}{\sigma},$$

which prevents information leakage from the future into the training process.

Preservation of Price Data

The adjusted closing prices $\mathbf{p}(t) \in R^N$ are left unscaled because:

1. the environment computes portfolio returns using price relatives $\frac{p_i(t+1)}{p_i(t)}$, which are scale-invariant,
2. the agent should operate in the true monetary domain when evaluating portfolio transitions,
3. scaling prices could introduce unrealistic distortions in relative volatility.

Thus the final preprocessed dataset consists of a concatenation:

$$\tilde{\mathbf{X}}(t) = [\tilde{\mathbf{x}}_{\text{sig}}(t), \mathbf{p}(t)].$$

Saving the Processed Datasets

To guarantee consistent experiment behavior and reproducibility, the scaled training and testing matrices are stored as

```
train_data_scaled.csv,    test_data_scaled.csv.
```

These files are subsequently loaded by the training and evaluation scripts, ensuring that every experimental run uses exactly the same preprocessed inputs.

6 Reinforcement Learning Environment

The trading environment is implemented in `portfolio_env.py` following the OpenAI Gym interface. This environment formulates asset allocation as a continuous-control task.

- **State Representation:** Each observation is the previously described 3D tensor of technical indicators over the recent time window.
- **Action Space:** The agent outputs a continuous vector of portfolio weights, one per stock. These weights are normalized to sum to one, with the residual implicitly treated as the cash allocation.
- **Portfolio Transition:** Given the allocation and next-day price relatives, the environment updates the portfolio value. Transaction costs are incorporated using an L_1 penalty proportional to the change in allocation.
- **Reward Function:** The reward at each step is set to the logarithmic portfolio return, $r_t = \log \left(\frac{V_{t+1}}{V_t} \right)$, which stabilizes training and reflects multiplicative portfolio growth.
- **Episode Logging:** At the end of each episode, detailed logs—including portfolio values, dates, rewards, and transaction costs—are exported to CSV files for interpretability and analysis.

7 Model Architecture

The reinforcement learning agent consists of two main components: a convolutional neural network (CNN) that processes the multi-asset temporal features, and a Proximal Policy Optimization (PPO) actor–critic architecture that learns a portfolio allocation policy. The CNN serves as a shared feature extractor for both the policy (actor) and value (critic) networks.

Convolutional Feature Extraction

At each decision time t , the agent receives an observation tensor

$$\mathcal{X}(t) \in R^{F \times W \times 1},$$

where F denotes the number of engineered features (e.g. 250), and W is the temporal window length (e.g. 30). The CNN applies a sequence of two-dimensional convolutions with kernels shaped to slide along the temporal dimension. If a convolutional kernel has shape $(k, 1)$, then for any feature channel f the operation effectively computes:

$$(\mathcal{X} * K)(f, \tau) = \sum_{i=0}^{k-1} K(i) \mathcal{X}(f, \tau + i),$$

allowing the network to detect short-term patterns such as momentum shifts, volatility spikes, or mean reversion tendencies within each feature independently.

Stacking several such convolutional layers yields an activation tensor

$$\mathcal{H}(t) \in R^{F' \times W' \times C},$$

where C is the number of channels and F', W' are reduced dimensions determined by the convolutional stride. This activation map is then flattened and passed through a fully connected layer to produce a fixed dimensional latent representation:

$$\mathbf{z}(t) \in R^d, \quad d = 128.$$

This vector $\mathbf{z}(t)$ serves as a compressed summary of current market conditions, integrating cross-asset and temporal information into a single feature vector.

Actor–Critic Policy Representation

The PPO framework uses the shared latent representation $\mathbf{z}(t)$ to parameterize two components:

- **Actor (policy network):** outputs a continuous vector of portfolio weights. Let the actor produce parameters $\boldsymbol{\mu}(t)$ and $\boldsymbol{\sigma}(t)$ of a multivariate normal distribution:

$$\pi_{\theta}(\mathbf{a}(t) \mid \mathcal{X}(t)) = \mathcal{N}(\boldsymbol{\mu}(t), \text{diag}(\boldsymbol{\sigma}(t)^2)),$$

where $\mathbf{a}(t)$ is the unnormalized allocation vector. A softmax or normalization step converts $\mathbf{a}(t)$ into valid portfolio weights that sum to one.

- **Critic (value network):** estimates the expected return from the latent state:

$$V_{\phi}(t) = V_{\phi}(\mathbf{z}(t)).$$

Both networks are trained jointly but maintain separate output layers. The shared CNN ensures that policy and value estimation rely on consistent temporal feature representations.

PPO Optimization Mechanism

During training, PPO updates the actor by maximizing the clipped surrogate objective:

$$\mathcal{L}^{\text{PPO}}(\theta) = E_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)],$$

where

$$r_t(\theta) = \frac{\pi_\theta(\mathbf{a}(t) \mid \mathcal{X}(t))}{\pi_{\theta_{\text{old}}}(\mathbf{a}(t) \mid \mathcal{X}(t))}, \quad A_t = \text{advantage estimate at time } t.$$

The latent vector $\mathbf{z}(t)$ produced by the CNN is treated as the state embedding for both the actor and critic. The critic is trained to minimize:

$$\mathcal{L}^{\text{value}}(\phi) = \frac{1}{2} (V_\phi(t) - R_t)^2,$$

where R_t is the empirical discounted return.

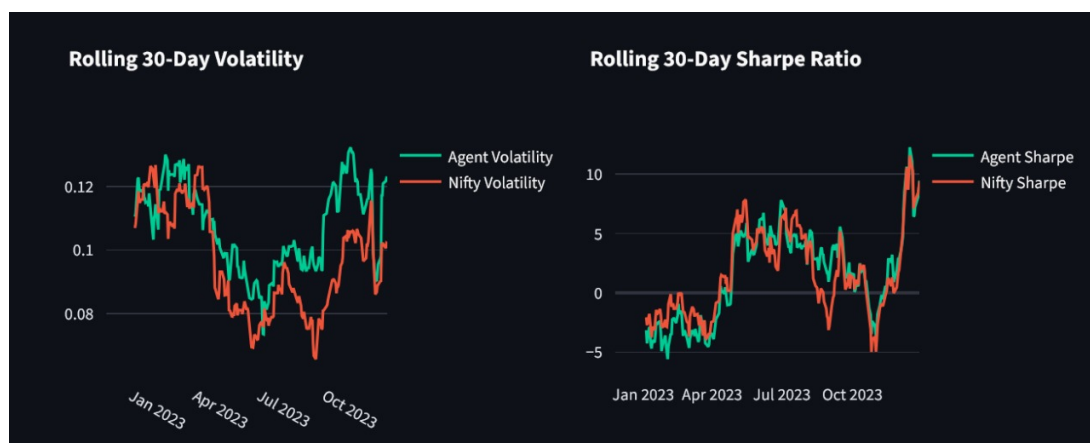
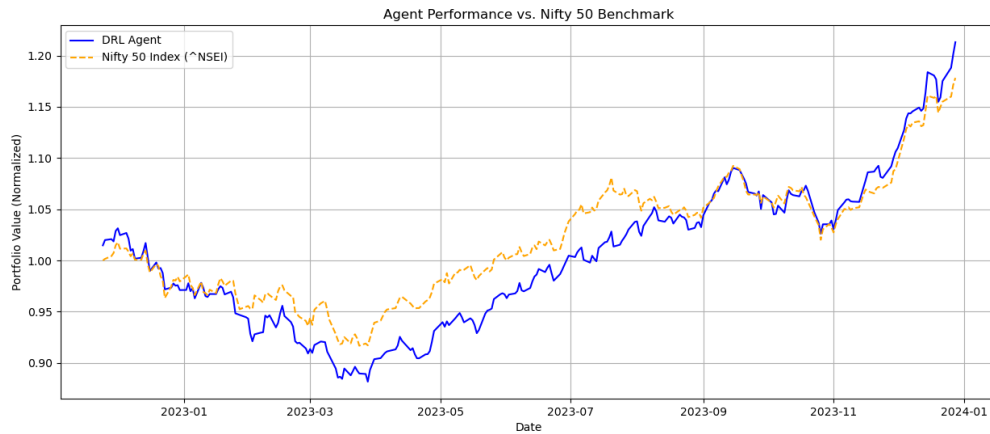
8 Training Procedure

The model functions as follows: the CNN interprets raw temporal patterns from dozens of assets, compressing them into a market state representation. The actor network uses this representation to propose portfolio allocations, while the critic assesses their long-term desirability. PPO then adjusts the policy in small, stable steps, ensuring that allocation decisions evolve in a controlled and statistically justified manner. Together, the CNN and PPO architecture enable the agent to learn allocation strategies that respond to evolving market structure in a stable and data-driven way.

The training loop is orchestrated. The script loads the training dataset, constructs the `PortfolioEnv`, and initializes the PPO agent with the custom CNN feature extractor. The agent is trained for the number of timesteps specified. PPO's clipped surrogate objective ensures stable updates by preventing excessively large policy shifts. During training, metrics are logged to TensorBoard, and the final model is saved to the path defined in the configuration file.

The trained PPO model is loaded and deployed deterministically in a test environment created from the held-out dataset. For each timestep, the script records the portfolio value and compares the agent's cumulative performance to that of the NIFTY 50 benchmark. Both series are normalized to a base value of 1 to enable direct comparison. The evaluation procedure also exports step-level logs, enabling computation of metrics such as returns, volatility, drawdowns, and turnover. The final plot closely resembles the Agent vs. NIFTY 50 performance chart provided in the project presentation.

8.1 Results and Observation



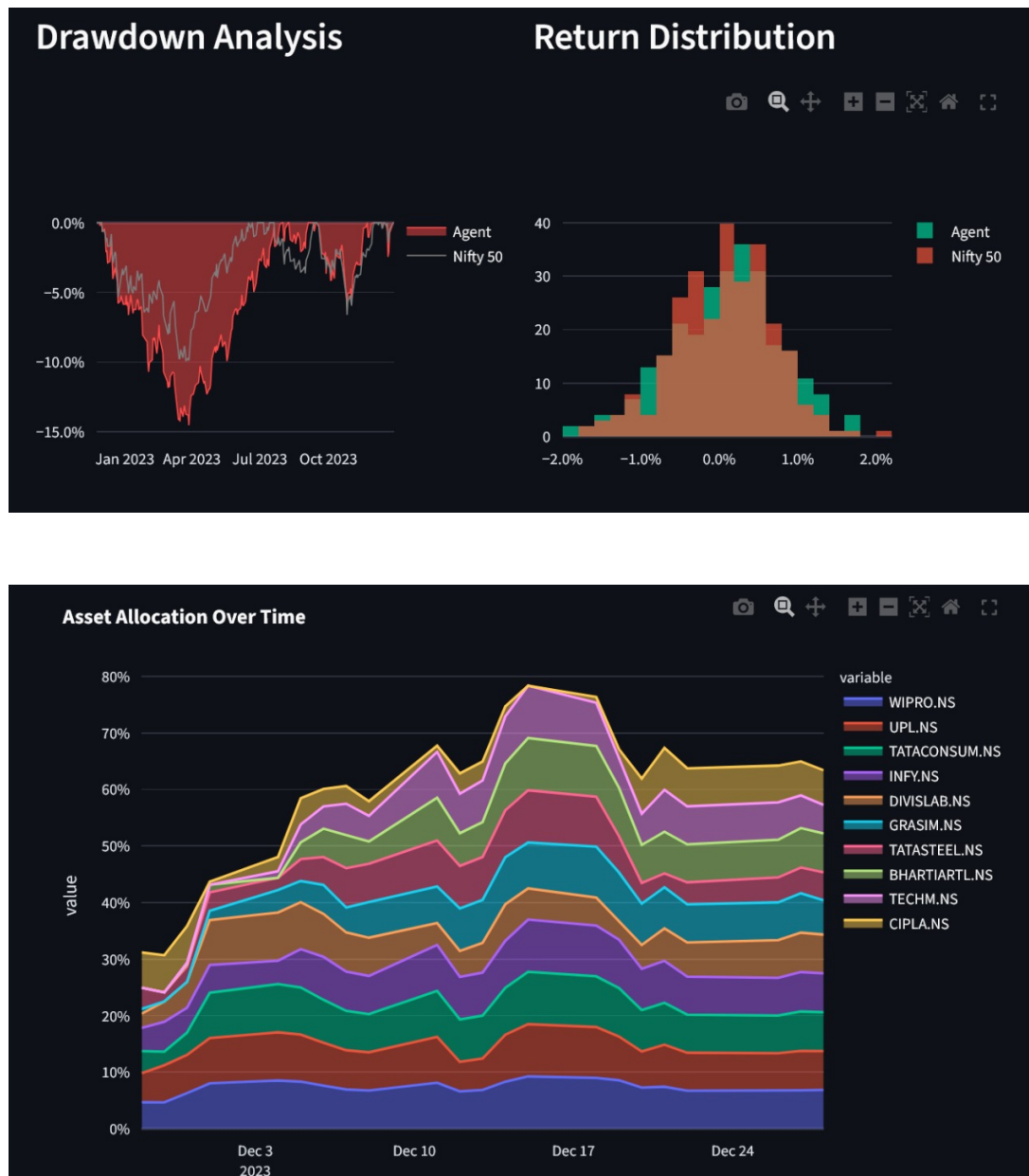


Figure 2: Results of the Deep Reinforcement Learning Asset Allocation System

9 Future Work

Although the proposed framework demonstrates promising results, several realistic extensions could significantly enhance its performance and applicability:

1. **Incorporation of Risk-Aware Objectives:** The current formulation focuses on maximizing log returns, which captures growth but does not explicitly control downside risk. Future work may incorporate risk-adjusted reward functions such as the Sharpe ratio, Conditional Value-at-Risk (CVaR), or drawdown penalties. Embedding such objectives into the reinforcement learning signal can encourage the agent to learn more stable, institutionally relevant allocation strategies that balance return and risk.
2. **More Realistic Market Simulation:** The trading environment assumes fixed transaction costs and frictionless execution. Practical deployment, however, must account for slippage, liquidity constraints, delayed order execution, and market-impact effects. Extending the environment to model these elements would produce more realistic portfolio trajectories and improve the robustness of the learned policy when transferred to live market conditions.



References

- [1] Thibaut Theate and Damien Ernst, *An Application of Deep Reinforcement Learning to Algorithmic Trading*, Montefiore Institute, University of Liege, Belgium.
- [2] Liu Jing and Yuncheol Kang, *Automated Cryptocurrency Trading Approach Using Ensemble Deep Reinforcement Learning: Learn to Understand Candlesticks*, Bigdata Analytics in Graduate School, Ewha Womans University, South Korea.
- [3] Vasileios Kochliaridis, Eleftherios Kouloumpiris, and Ioannis Vlahavas, *Combining Deep Reinforcement Learning with Technical Analysis and Trend Monitoring on Cryptocurrency Markets*, Published: 20 April 2023.
- [4] Berend Jelmer Dirk Gort, Xiao-Yang Liu, Jiechao Gao, Shuaiyu Chen, and Christina Dan Wang, *Deep Reinforcement Learning for Cryptocurrency Trading: Practical Approach to Address Backtest Overfitting*.
- [5] Zihao Zhang, Stefan Zohren, and Stephen Roberts, *Deep Reinforcement Learning for Trading*, Department of Engineering Science, Oxford-Man Institute of Quantitative Finance, University of Oxford.
- [6] Hamidreza Ghadiri and Ehsan Hajizadeh, *Designing a Cryptocurrency Trading System with Deep Reinforcement Learning Utilizing LSTM Neural Networks and XGBoost Feature Selection*, Department of Industrial Engineering and Management Systems, Amirkabir University of Technology, Iran.